LAPORAN PRAKTIKUM PEMROGRAMAN WEB

Detail Produk, Riwayat Penjualan, dan Transaksi Point of Sale

Panduan langkah demi langkah implementasi Pertemuan 11 dan 12 pada aplikasi POS Barokah Mart

Laravel 13 · Blade · Alpine.js · Tailwind CSS

Nama	DeandraPutra Yuriansyah
NPM	2410631170133
Mata kuliah	Pemrograman Web (FPW)
Studi kasus	POS Barokah Mart
Framework	Laravel 13.29
Database	MySQL / MariaDB
Tanggal	8 November 2026

Disusun oleh **DeandraPutra Yuriansyah** (NPM 2410631170133). Dokumen ini disusun berdasarkan implementasi nyata pada proyek **C:\xampp\htdocs\FPW\FPW_Test1**. Seluruh potongan kode di dalamnya diambil langsung dari berkas proyek yang sudah diuji.

DAFTAR ISI

Isi Dokumen

1 Pendahuluan: tujuan, prasyarat, dan hasil akhir

2 Pertemuan 11 — Detail Produk dan Riwayat Penjualan

3 Pertemuan 12 — Transaksi Penjualan (Point of Sale)

4 Kesimpulan dan checklist pengujian

5 Pemecahan masalah (troubleshooting)

6 Daftar berkas yang diubah dan dibuat

7 Referensi

CARA MEMBACA DOKUMEN

Setiap bab disusun mengikuti alur praktikum: konsep singkat, potongan kode yang dipakai, penjelasan baris penting, lalu langkah pengujian. Kode yang ditampilkan adalah kode final yang ada di proyek, bukan pseudocode.

BAB 1

Pendahuluan

Konteks, prasyarat, dan hasil yang akan dicapai

1.1 Tujuan pembelajaran

- Membuat halaman detail produk menggunakan **Route Model Binding**.
- Menampilkan relasi bertingkat (*nested relation*) dan agregasi sederhana (`sum` , `count`) dengan Eloquent.
- Membangun layar kasir interaktif dengan **Alpine.js**: pencarian produk, keranjang, total otomatis, dan kembalian.
- Menyimpan transaksi penjualan secara atomik memakai `DB::transaction()` dan `lockForUpdate()` .
- Menghasilkan struk transaksi dan memastikan stok berkurang dengan benar.

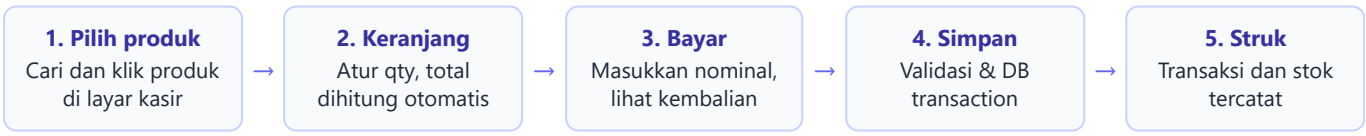
1.2 Prasyarat

- Pertemuan 1–10 sudah selesai: autentikasi, otorisasi berbasis peran, layout Blade, data master kategori/produk, serta pencarian dan pagination.
- PHP 8.3 atau lebih baru, Composer, Node.js, dan MySQL/MariaDB sudah berjalan.
- Dependensi frontend terpasang (`node_modules`) dan aset Vite sudah dibangun.
- Database `laravel` berisi tabel `users` , `categories` , `products` , `transactions` , dan `transaction_details` .

1.3 Hasil akhir

Setelah kedua pertemuan diterapkan, aplikasi memiliki dua kemampuan utama:

1. **Halaman detail produk** yang menampilkan harga, stok, total unit terjual, dan sepuluh riwayat penjualan terakhir untuk produk tersebut.
2. **Layar kasir (POS)** yang memungkinkan kasir memilih produk, mengatur jumlah, menghitung total dan kembalian secara langsung, lalu menyimpan transaksi beserta detail dan pengurangan stok dalam satu transaksi database.



1.4 Ringkasan perubahan berkas

Berkas	Status	Peran
`app/Http/Controllers/ProductController.php`	Diubah	Search, filter, pagination, detail produk, update, delete
`app/Http/Requests/UpdateProductRequest.php`	Diubah	Validasi update dengan pengecualian unique
`resources/views/master-data/product/index.blade.php`	Diubah	Form pencarian, filter kategori, tombol Detail/Ubah/Hapus
`resources/views/master-data/product/edit.blade.php`	Diubah	Form ubah produk lengkap
`resources/views/master-data/product/show.blade.php`	Dibuat	Halaman detail dan riwayat penjualan
`app/Http/Controllers/PosController.php`	Diubah	Layar kasir, simpan transaksi, struk
`app/Http/Requests/StorePosRequest.php`	Dibuat	Validasi array item keranjang
`resources/views/pos/index.blade.php`	Dibuat	UI keranjang dengan Alpine.js
`resources/views/pos/struk.blade.php`	Dibuat	Halaman struk transaksi
`resources/js/app.js`	Diubah	Inisialisasi Alpine.js
`resources/css/app.css`	Diubah	Gaya cetak struk
`routes/web.php`	Diubah	Rute POS dan struk
`package.json` / `package-lock.json`	Diubah	Dependensi alpinejs

BAB 2 · PERTEMUAN 11

Detail Produk dan Riwayat Penjualan

Route Model Binding, eager loading bertingkat, dan agregasi Eloquent

2.1 Konsep kunci

Halaman detail memanfaatkan rute `GET /products/{product}` yang sudah dibuat otomatis oleh `Route::resource('products', ProductController::class)` pada Pertemuan 7. Karena nama parameter rute sama dengan nama model, Laravel melakukan **Route Model Binding**: angka pada URL dicari ke tabel `products`, lalu objek `Product` langsung disuntikkan ke method. Jika data tidak ditemukan, Laravel otomatis mengembalikan 404 tanpa kode tambahan.

Untuk menampilkan riwayat, kita memuat relasi bertingkat `transactionDetails.transaction`. Tanda titik berarti: ambil semua detail transaksi milik produk ini, lalu untuk setiap detail ambil transaksi induknya. Setelah relasi dimuat, `sum('qty')` dihitung di memori (koleksi), bukan query baru.

> **WITH() VS LOAD()**
>
> `with()` dipakai saat query builder baru dibentuk. `load()` dipakai saat model sudah terlanjur diambil, misalnya lewat Route Model Binding, tetapi relasinya belum dimuat. Di method `show()` kita memakai `load()` karena `$product` sudah disuntikkan Laravel.

2.2 Method show() pada ProductController

Tambahkan method berikut. Perhatikan bahwa `$product->transactionDetails` (tanpa tanda kurung) mengambil koleksi yang sudah dimuat, sedangkan `$product->transactionDetails()` (dengan tanda kurung) membuat query baru agar bisa ditambah `latest()` dan `take(10)`.

```php
public function show(Product $product)
{
    $product->load(['transactionDetails.transaction']);

    $totalTerjual = $product->transactionDetails->sum('qty');

    $riwayat = $product->transactionDetails()
        ->with('transaction')
        ->latest()
        ->take(10)
        ->get();

    return view('master-data.product.show', compact('product', 'totalTerjual', 'riwayat'));
}
```

Penjelasan baris penting

- `$product->load(['transactionDetails.transaction'])` memuat dua tingkat relasi sekaligus, sehingga Blade tidak memicu query tambahan di dalam loop.
- `$product->transactionDetails->sum('qty')` menjumlahkan kolom `qty` dari koleksi yang sudah ada di memori.
- `transactionDetails()->with('transaction')->latest()->take(10)->get()` mengambil sepuluh detail terbaru beserta transaksi induknya.

2.3 Membuat view detail

Buat berkas `resources/views/master-data/product/show.blade.php`. Struktur halaman terdiri dari kartu ringkasan (harga, stok, total terjual) dan tabel riwayat. Blok `@forelse ... @empty` menampilkan pesan "Belum pernah terjual" ketika relasi masih kosong.

```
<x-app-layout>
    <x-slot name="header">
        <div class="flex justify-between items-center">
            <h2 class="font-semibold text-xl text-gray-800">Detail Produk</h2>
            <div class="space-x-4 text-sm">
                <a href="{{ route('products.index') }}" class="text-gray-500">Kembali</a>
                <a href="{{ route('products.edit', $product) }}" class="text-indigo-600">Ubah Produk</a>
            </div>
        </div>
    </x-slot>

    <div class="py-12 space-y-6">
        <div class="max-w-4xl mx-auto sm:px-6 lg:px-8">
            <x-card>
                <h3 class="text-lg font-semibold">{{ $product->name }}</h3>
                <p class="text-sm text-gray-500 mb-4">
                    {{ $product->code }} &middot; {{ $product->category->name }}
                </p>

                <div class="grid grid-cols-3 gap-4 mb-6">
                    <div class="bg-gray-50 rounded-md p-4 text-center">
                        <p class="text-xs text-gray-500">Harga</p>
                        <p class="text-lg font-semibold">
                            Rp{{ number_format($product->price, 0, ',', '.') }}
                        </p>
                    </div>
                    <div class="bg-gray-50 rounded-md p-4 text-center">
                        <p class="text-xs text-gray-500">Stok Saat Ini</p>
                        <p class="text-lg font-semibold">{{ $product->stock }} {{ $product->unit }}</p>
                    </div>
                    <div class="bg-gray-50 rounded-md p-4 text-center">
                        <p class="text-xs text-gray-500">Total Terjual</p>
                        <p class="text-lg font-semibold">{{ $totalTerjual }} {{ $product->unit }}</p>
                    </div>
                </div>

                <h4 class="font-medium mb-2">Riwayat Penjualan Terakhir</h4>
                <table class="min-w-full divide-y divide-gray-200 text-sm">
                    <thead>
                        <tr>
                            <th class="px-3 py-2 text-left">No. Invoice</th>
                            <th class="px-3 py-2 text-left">Tanggal</th>
                            <th class="px-3 py-2 text-right">Qty</th>
                            <th class="px-3 py-2 text-right">Subtotal</th>
                        </tr>
                    </thead>
                    <tbody class="divide-y divide-gray-100">
                        @forelse ($riwayat as $item)
                            <tr>
                                <td class="px-3 py-2">{{ $item->transaction->invoice_number }}</td>
                                <td class="px-3 py-2">
                                    {{ $item->transaction->created_at->format('d M Y H:i') }}
                                </td>
                                <td class="px-3 py-2 text-right">{{ $item->qty }}</td>
                                <td class="px-3 py-2 text-right">
                                    Rp{{ number_format($item->subtotal, 0, ',', '.') }}
```

```
                            </td>
                        </tr>
                    @empty
                        <tr>
                            <td colspan="4" class="px-3 py-4 text-center text-gray-500">
                                Belum pernah terjual.
                            </td>
                        </tr>
                    @endforelse
                </tbody>
            </table>
        </x-card>
    </div>
</div>
</x-app-layout>
```

DATA RIWAYAT MASIH KOSONG?

Itu normal sebelum Pertemuan 12 selesai. Tabel `transaction_details` baru terisi setelah kasir menyimpan transaksi lewat layar POS. Halaman detail sudah siap menampilkan data begitu transaksi pertama dibuat.

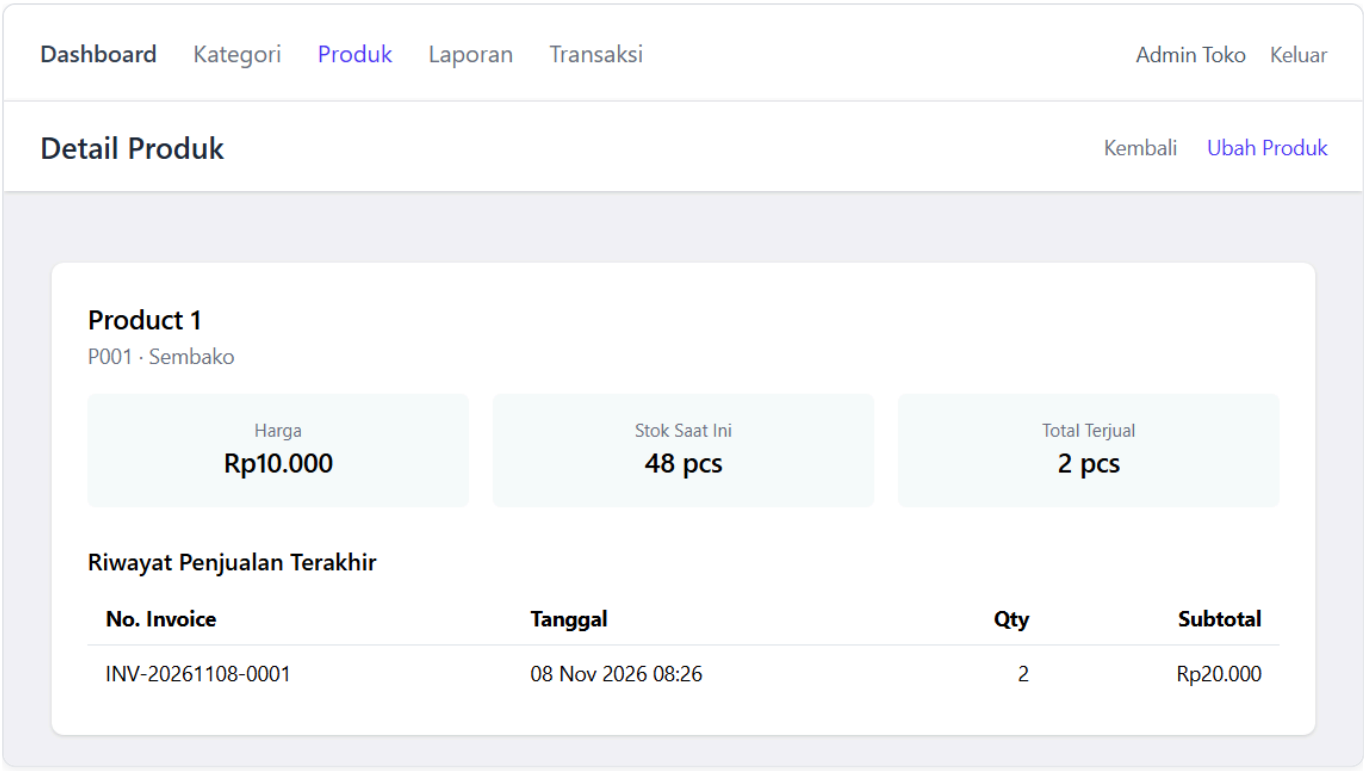
2.4 Menambahkan tautan Detail di daftar produk

Pada `resources/views/master-data/product/index.blade.php`, tambahkan tautan berikut di kolom aksi setiap baris produk:

```
<td class="px-4 py-2 text-sm text-center space-x-2 whitespace-nowrap">
    <a href="{{ route('products.show', $product) }}" class="text-gray-600">Detail</a>
    <a href="{{ route('products.edit', $product) }}" class="text-indigo-600">Ubah</a>
    <!-- tombol Hapus ... -->
</td>
```

2.5 Menguji halaman detail

Masuk sebagai admin, buka menu **Produk**, lalu klik **Detail** pada salah satu produk. Halaman akan menampilkan ringkasan dan riwayat penjualan seperti tangkapan layar berikut.



Gambar 2.1 — Halaman detail produk: harga, stok, total terjual, dan riwayat penjualan.

Perhatikan bahwa **Total Terjual** dihitung dari seluruh detail transaksi, sedangkan tabel di bawahnya hanya menampilkan sepuluh transaksi terakhir. Harga dan subtotal di riwayat berasal dari `transaction_details`, sehingga perubahan harga produk di kemudian hari tidak mengubah nilai transaksi lama.

BAB 3 · PERTEMUAN 12

Transaksi Penjualan (Point of Sale)

Keranjang Alpine.js, validasi server, DB transaction, dan struk

3.1 Alur kerja layar kasir

Layar kasir adalah jantung aplikasi POS. Kasir memilih produk, sistem menghitung total, kasir memasukkan nominal bayar, lalu sistem menghitung kembalian. Saat tombol **Simpan Transaksi** ditekan, server memvalidasi ulang seluruh data, mengunci baris produk, mengurangi stok, membuat satu baris `transactions`, dan membuat banyak baris `transaction_details` — semuanya dalam satu transaksi database.

MENGAPA HARUS ATOMIK?

Bayangkan transaksi berhasil dibuat tetapi pengurangan stok gagal karena koneksi putus. Data menjadi tidak konsisten: barang tercatat terjual, tetapi stok tidak berkurang. `DB::transaction()` memastikan seluruh operasi berhasil atau seluruhnya dibatalkan (rollback).

3.2 Rute POS

Daftarkan tiga rute di dalam grup middleware `auth` dan `role:admin,kasir` pada `routes/web.php`. Rute struk memakai Route Model Binding `{transaction}`, pola yang sama seperti `{product}`.

```
Route::middleware(['auth', 'role:admin,kasir'])->group(function () {
    Route::get('/pos', [PosController::class, 'index'])->name('pos.index');
    Route::post('/pos', [PosController::class, 'store'])->name('pos.store');
    Route::get('/pos/{transaction}/struk', [PosController::class, 'struk'])->name('pos.struk');
});
```

Method	URL	Nama rute	Fungsi
GET	/pos	pos.index	Menampilkan layar kasir dan daftar produk berstok
POST	/pos	pos.store	Menyimpan transaksi dan mengurangi stok
GET	/pos/{transaction}/struk	pos.struk	Menampilkan struk transaksi

3.3 Controller: mengirim produk ke layar kasir

Method `index()` hanya mengambil produk yang stoknya lebih dari nol. Kolom yang diambil dibatasi agar payload JSON ringan. Produk dengan stok habis sengaja tidak dikirim supaya kasir tidak bisa memilih barang yang memang tidak tersedia.

```
    public function index()
    {
        $products = Product::where('stock', '>', 0)
            ->orderBy('name')
            ->get(['id', 'code', 'name', 'price', 'stock', 'unit']);

        return view('pos.index', compact('products'));
    }
```

3.4 Memasang Alpine.js

Proyek ini tidak memakai starter kit, jadi Alpine.js dipasang manual lewat npm. Jalankan perintah berikut di root proyek:

```
npm install alpinejs
npm run build
```

Lalu inisialisasi Alpine pada `resources/js/app.js`. Berkas ini sudah dimuat oleh `@vite(['resources/css/app.css', 'resources/js/app.js'])` di layout, sehingga tidak perlu menambahkan tag script baru di setiap halaman.

```
import Alpine from 'alpinejs';

window.Alpine = Alpine;

Alpine.start();
```

MENGAPA ALPINE.JS?

Alpine cukup ringan untuk mengelola state keranjang (daftar item, qty, total, kembalian) langsung di Blade, tanpa harus membangun aplikasi Single Page terpisah. Atribut `x-data`, `x-for`, dan `x-model` membuat reaktivitas tetap sederhana.

3.5 State dan fungsi keranjang

Seluruh state keranjang didefinisikan dalam fungsi `posCart(products)`. Data produk dari server dikirim sebagai JSON lewat `$products->toJson()`, lalu menjadi nilai awal `x-data`.

```
function posCart(products) {
    return {
        products: products,
        cart: [],
        keyword: '',
        pay: 0,

        filteredProducts() {
            if (!this.keyword) return this.products;
            const k = this.keyword.toLowerCase();
            return this.products.filter(p =>
                p.name.toLowerCase().includes(k) ||
                p.code.toLowerCase().includes(k)
            );
        },

        addItem(product) {
            const existing = this.cart.find(i => i.id === product.id);
            if (existing) {
                if (existing.qty < product.stock) existing.qty++;
                return;
            }
            this.cart.push({ ...product, qty: 1 });
        },

        clampQty(item) {
            if (item.qty < 1) item.qty = 1;
            if (item.qty > item.stock) item.qty = item.stock;
```

```
                },

                removeItem(index) {
                    this.cart.splice(index, 1);
                },

                total() {
                    return this.cart.reduce((sum, i) => sum + (i.price * i.qty), 0);
                },

                change() {
                    return Math.max(this.pay - this.total(), 0);
                },

                formatRupiah(value) {
                    return 'Rp' + Number(value || 0).toLocaleString('id-ID');
                },
            }
        }
```

Peran setiap fungsi

- `filteredProducts()` menyaring produk berdasarkan nama atau kode secara instan di sisi klien, tanpa reload halaman.
- `addItem()` menambah produk baru atau menaikkan qty jika produk sudah ada di keranjang; qty tidak boleh melebihi stok.
- `clampQty()` menjaga qty tetap di antara 1 dan stok.
- `total()` menjumlahkan `price * qty` untuk seluruh item.
- `change()` menghitung kembalian dan tidak pernah negatif.
- `formatRupiah()` memformat angka dengan pemisah ribuan `id-ID`.

3.6 Form keranjang dan input tersembunyi

Bagian terpenting untuk controller adalah dua input tersembunyi di dalam loop keranjang: `items[index][product_id]` dan `items[index][qty]`. Sintaks tersebut membuat HTML mengirim array asosiatif; Laravel menerimanya sebagai `$request->items` berisi daftar `product_id` dan `qty`.

```
                    <form action="{{ route('pos.store') }}" method="POST">
                        @csrf

                        <template x-for="(item, index) in cart" :key="item.id">
                            <div class="flex items-center justify-between mb-2 text-sm">
                                <div class="flex-1">
                                    <p x-text="item.name"></p>
                                    <p class="text-xs text-gray-500" x-text="formatRupiah(item.price) + ' / ' +
item.unit"></p>
                                </div>
                                <input type="number" min="1" :max="item.stock" x-model.number="item.qty"
                                    @input="clampQty(item)" class="w-16 rounded-md border-gray-300 text-center">
                                <button type="button" @click="removeItem(index)" class="text-red-500 ml-2">&times;
</button>

                                <input type="hidden" :name="'items[' + index + '][product_id]'" :value="item.id">
                                <input type="hidden" :name="'items[' + index + '][qty]'" :value="item.qty">
                            </div>
                        </template>

                        <p x-show="cart.length === 0" class="text-sm text-gray-500 py-3">
                            Keranjang masih kosong. Klik produk di sebelah kiri.
                        </p>
```

```
                        <div class="border-t pt-3 mt-3 space-y-1">
                            <div class="flex justify-between font-semibold">
                                <span>Total</span>
                                <span x-text="formatRupiah(total())"></span>
                            </div>

                            <label class="block text-sm mt-2">Bayar</label>
                            <input type="number" name="pay" x-model.number="pay" min="0"
                                class="w-full rounded-md border-gray-300">

                            <div class="flex justify-between text-sm mt-1">
                                <span>Kembalian</span>
                                <span x-text="formatRupiah(change())"></span>
                            </div>
                        </div>

                        <button type="submit" :disabled="cart.length === 0 || pay < total()"
                            class="w-full mt-4 bg-indigo-600 text-white py-2 rounded-md disabled:opacity-50">
                            Simpan Transaksi
                        </button>
                    </form>
```

VALIDASI SISI KLIEN BUKAN PENGAMAN

Tombol submit dinonaktifkan jika keranjang kosong atau nominal bayar kurang dari total. Itu hanya kenyamanan pengguna. Data dari browser tetap harus divalidasi ulang di server, karena klien tidak pernah bisa dipercaya sepenuhnya.

3.7 Validasi server dengan StorePosRequest

Buat Form Request baru dengan perintah `php artisan make:request StorePosRequest`, lalu isi aturannya. Wildcard `items.*.product_id` memeriksa setiap elemen array, bukan hanya elemen pertama.

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePosRequest extends FormRequest
{
    /**
     * Admin dan kasir boleh menyimpan transaksi penjualan.
     */
    public function authorize(): bool
    {
        return auth()->check()
            && in_array(auth()->user()->role, ['admin', 'kasir'], true);
    }

    /**
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'items' => 'required|array|min:1',
            'items.*.product_id' => 'required|exists:products,id',
            'items.*.qty' => 'required|integer|min:1',
            'pay' => 'required|integer|min:0',
        ];
    }
```

```
    public function messages(): array
    {
        return [
            'items.required' => 'Keranjang belanja masih kosong.',
            'items.min' => 'Pilih minimal satu produk untuk ditransaksikan.',
            'items.*.product_id.exists' => 'Produk yang dipilih tidak ditemukan.',
            'items.*.qty.min' => 'Jumlah produk minimal 1.',
            'pay.required' => 'Nominal pembayaran wajib diisi.',
        ];
    }
}
```

3.8 Menyimpan transaksi secara atomik

Method `store()` membungkus seluruh operasi dalam `DB::transaction()`. Perhatikan tiga hal penting:

1. **lockForUpdate()** mengunci baris produk sampai transaksi selesai, mencegah dua kasir menjual unit terakhir secara bersamaan.
2. **Exception** di dalam closure memicu rollback otomatis, termasuk pengurangan stok yang sudah terjadi pada iterasi sebelumnya.
3. **Nomor invoice** dibentuk dari tanggal dan urutan transaksi harian, misalnya `INV-20261108-0001`.

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\StorePosRequest;
use App\Models\Product;
use App\Models\Transaction;
use Illuminate\Support\Facades\DB;

class PosController extends Controller
{
    /**
     * Pertemuan 12: tampilkan produk yang stoknya masih tersedia.
     */
    public function index()
    {
        $products = Product::where('stock', '>', 0)
            ->orderBy('name')
            ->get(['id', 'code', 'name', 'price', 'stock', 'unit']);

        return view('pos.index', compact('products'));
    }

    /**
     * Pertemuan 12: simpan transaksi penjualan secara atomik.
     */
    public function store(StorePosRequest $request)
    {
        try {
            $transaction = DB::transaction(function () use ($request) {
                $total = 0;
                $details = [];

                foreach ($request->items as $item) {
                    // Kunci baris produk agar stok tidak dibaca ganda oleh kasir lain.
                    $product = Product::lockForUpdate()->findOrFail($item['product_id']);

                    if ($product->stock < $item['qty']) {
```

```
                    throw new \Exception("Stok {$product->name} tidak mencukupi.");
                }

                $subtotal = $product->price * $item['qty'];
                $total += $subtotal;

                $details[] = [
                    'product_id' => $product->id,
                    'qty' => $item['qty'],
                    'price' => $product->price,
                    'subtotal' => $subtotal,
                ];

                $product->decrement('stock', $item['qty']);
            }

            if ($request->pay < $total) {
                throw new \Exception('Nominal bayar kurang dari total belanja.');
            }

            $trx = Transaction::create([
                'invoice_number' => 'INV-' . now()->format('Ymd') . '-' . str_pad(
                    Transaction::whereDate('created_at', now())->count() + 1,
                    4,
                    '0',
                    STR_PAD_LEFT
                ),
                'user_id' => auth()->id(),
                'total' => $total,
                'pay' => $request->pay,
                'change' => $request->pay - $total,
            ]);

            $trx->details()->createMany($details);

            return $trx;
        });
    } catch (\Exception $e) {
        return redirect()->route('pos.index')
            ->with('error', $e->getMessage())
            ->withInput();
    }

    return redirect()->route('pos.struk', $transaction)
        ->with('success', 'Transaksi berhasil disimpan.');
}

/**
 * Pertemuan 12: halaman struk sebagai konfirmasi transaksi.
 */
public function struk(Transaction $transaction)
{
    $transaction->load('details.product', 'user');

    return view('pos.struk', compact('transaction'));
}
}
```

KENAPA TRY/CATCH?

Modul asli melempar exception untuk stok kurang. Pada implementasi ini exception ditangkap agar kasir kembali ke layar POS dengan pesan yang ramah, bukan halaman error 500. Rollback tetap terjadi karena exception dilempar di dalam closure `DB::transaction()`.

Ilustrasi rollback

Misalnya keranjang berisi Produk A (qty 1) dan Produk B (qty 5), tetapi stok B hanya 2. Iterasi pertama sudah mengurangi stok A. Ketika iterasi kedua mendeteksi stok B kurang, exception dilempar. Laravel membatalkan pengurangan stok A dan tidak pernah membuat baris transaksi. Stok kembali seperti semula.

3.9 Halaman struk

Setelah transaksi tersimpan, pengguna diarahkan ke `pos.struk`. Method `struk()` memuat relasi `details.product` dan `user` sekaligus agar struk bisa dicetak tanpa query tambahan di dalam loop.

```
    public function struk(Transaction $transaction)
    {
        $transaction->load('details.product', 'user');

        return view('pos.struk', compact('transaction'));
    }
```

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl">Struk Transaksi</h2>
    </x-slot>

    <div class="py-8">
        <div class="max-w-md mx-auto sm:px-6">
            <x-card>
                @if (session('success'))
                    <div class="mb-4 p-3 bg-green-100 text-green-700 rounded-md text-sm no-print">
                        {{ session('success') }}
                    </div>
                @endif

                <p class="text-center font-semibold">POS Barokah Mart</p>
                <p class="text-center text-xs text-gray-500 mb-3">{{ $transaction->invoice_number }}</p>
                <p class="text-xs text-gray-500 mb-3">
                    {{ $transaction->created_at->format('d M Y H:i') }} &middot;
                    Kasir: {{ $transaction->user->name }}
                </p>

                <hr class="mb-3">

                @foreach ($transaction->details as $detail)
                    <div class="flex justify-between text-sm mb-1">
                        <span>{{ $detail->product->name }} x{{ $detail->qty }}</span>
                        <span>Rp{{ number_format($detail->subtotal, 0, ',', '.') }}</span>
                    </div>
                @endforeach

                <hr class="my-3">

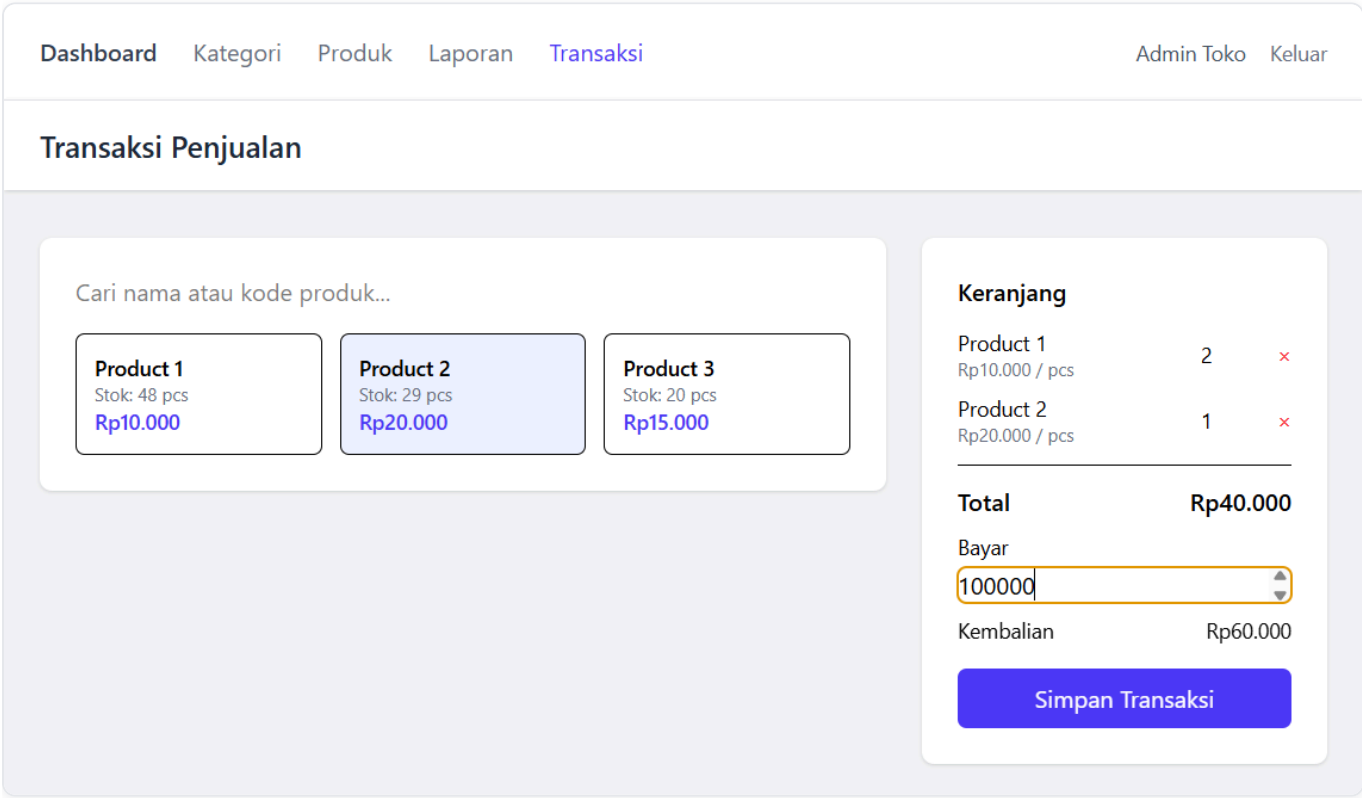
                <div class="flex justify-between font-semibold">
                    <span>Total</span>
                    <span>Rp{{ number_format($transaction->total, 0, ',', '.') }}</span>
```

```
                </div>
                <div class="flex justify-between text-sm">
                    <span>Bayar</span>
                    <span>Rp{{ number_format($transaction->pay, 0, ',', '.') }}</span>
                </div>
                <div class="flex justify-between text-sm">
                    <span>Kembali</span>
                    <span>Rp{{ number_format($transaction->change, 0, ',', '.') }}</span>
                </div>

                <div class="mt-4 space-y-2 no-print">
                    <button type="button" onclick="window.print()"
                        class="w-full border border-indigo-600 text-indigo-600 py-2 rounded-md">
                        Cetak Struk
                    </button>
                    <a href="{{ route('pos.index') }}"
                        class="block text-center bg-indigo-600 text-white py-2 rounded-md">
                        Transaksi Baru
                    </a>
                </div>
            </x-card>
        </div>
    </div>
</x-app-layout>
```

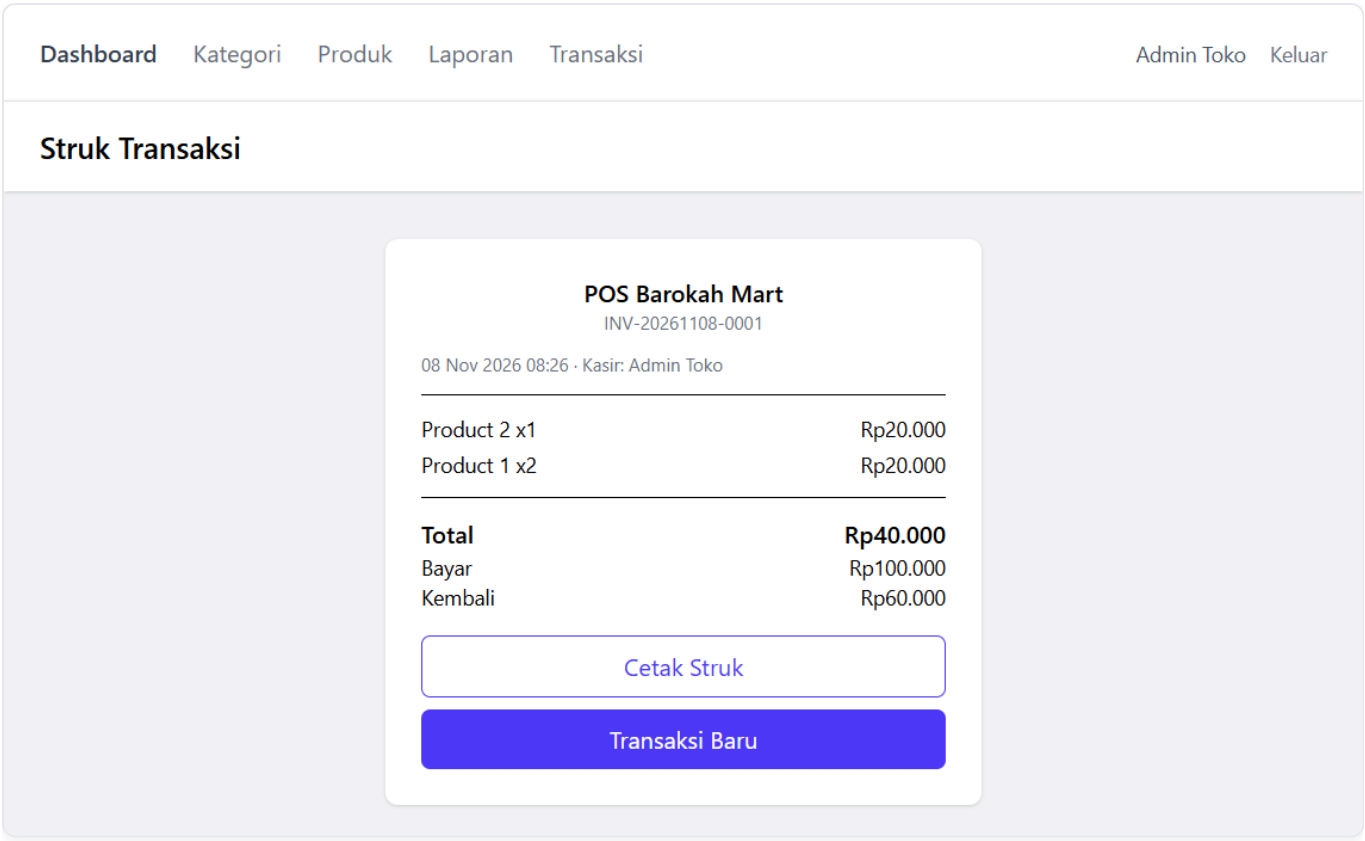
3.10 Menguji layar kasir

Masuk sebagai admin atau kasir, buka menu **Transaksi**, lalu klik beberapa produk. Total dan kembalian harus berubah secara langsung tanpa reload.



Gambar 3.1 — Layar kasir: pencarian produk, keranjang, total otomatis, dan kembalian.

Setelah tombol **Simpan Transaksi** ditekan, sistem menampilkan struk seperti berikut. Perhatikan nomor invoice, daftar item, total, bayar, dan kembalian.



Gambar 3.2 — Struk transaksi yang dihasilkan setelah penyimpanan berhasil.

3.11 Hasil pengujian

Skenario	Hasil yang diamati	Status
Login admin	Diarahkan ke dashboard	Berhasil
Pencarian “Product” + filter kategori Sembako	Menampilkan Product 1	Berhasil
Buka detail Product 1	Total terjual dan riwayat tampil	Berhasil
Checkout 2× Product 1 + 1× Product 2, bayar Rp100.000	Total Rp40.000, kembalian Rp60.000, stok berkurang	Berhasil
Beli 999 unit (stok tidak cukup)	Pesan “Stok Product 1 tidak mencukupi.”, stok tidak berubah	Berhasil
Hapus produk yang sudah punya riwayat	Pesan produk tidak dapat dihapus	Berhasil
Ubah harga produk	Pesan “Produk berhasil diperbarui.”	Berhasil

CATATAN PENGUJIAN

Data transaksi yang dibuat saat pengujian kemudian dihapus dan stok dikembalikan ke nilai semula, sehingga database tetap bersih. Anda dapat membuat transaksi baru kapan saja melalui layar POS.

3.12 Catatan penting untuk pengembangan

- **Race condition nomor invoice.** Pola `count() + 1` cukup untuk praktikum, tetapi pada trafik tinggi dua transaksi bisa memperoleh nomor yang sama. Untuk produksi, gunakan sequence database, UUID, atau kunci tambahan.

- **Soft delete.** Produk yang sudah memiliki riwayat transaksi sebaiknya tidak dihapus permanen. Pertimbangkan `SoftDeletes` agar nama produk lama tetap muncul di struk.
- **Otorisasi.** Rute POS dibatasi untuk admin dan kasir. Pastikan middleware `role` sudah terdaftar di `bootstrap/app.php`.

BAB 4

Kesimpulan dan Checklist

4.1 Kesimpulan

Pertemuan 11 melengkapi siklus data master dengan halaman detail yang informatif. Route Model Binding menghilangkan pencarian manual, eager loading bertingkat menjaga performa, dan agregasi koleksi menyajikan total terjual tanpa query berulang.

Pertemuan 12 menyatukan seluruh fitur sebelumnya menjadi transaksi penjualan yang utuh. Alpine.js menangani interaksi keranjang di sisi klien, sedangkan `DB::transaction()` dan `lockForUpdate()` menjaga konsistensi data di sisi server. Hasilnya, stok, transaksi, dan struk selalu selaras meskipun terjadi kegagalan di tengah proses.

4.2 Checklist implementasi

- ✓ Method `show()` memuat relasi dan menghitung total terjual.
- ✓ View `show.blade.php` menampilkan ringkasan dan riwayat.
- ✓ Tautan **Detail** tersedia di daftar produk.
- ✓ Rute `pos.index`, `pos.store`, dan `pos.struk` terdaftar dalam middleware yang benar.
- ✓ Alpine.js terpasang dan `Alpine.start()` dipanggil.
- ✓ Keranjang mengirim array `items` yang valid.
- ✓ `StorePosRequest` memvalidasi setiap item.
- ✓ Transaksi dan detail disimpan di dalam `DB::transaction()`.
- ✓ Stok berkurang dan struk menampilkan data yang benar.
- ✓ Kasus stok kurang dan hapus produk berelasi ditangani dengan pesan yang jelas.

BAB 5

Pemecahan Masalah

5.1 “Vite manifest not found” atau tampilan tanpa CSS

Aset belum dibangun. Jalankan `npm install` lalu `npm run build`. Jika sebelumnya memakai `npm run dev`, pastikan server Vite masih berjalan atau hapus berkas `public/hot` agar Laravel memakai hasil build.

5.2 Alpine.js tidak berfungsi

- Pastikan `alpinejs` ada di `package.json`.
- Pastikan `resources/js/app.js` mengimpor dan menjalankan Alpine.
- Jalankan `npm run build`, lalu muat ulang halaman dengan `Ctrl` + `F5`.
- Periksa Console browser untuk pesan error JavaScript.

5.3 Error 419 Page Expired

Token CSRF hilang atau kedaluwarsa. Pastikan setiap form memiliki `@csrf` dan session driver berjalan normal. Jika baru mengubah konfigurasi, bersihkan cache dengan `php artisan optimize:clear`.

5.4 Error 403 Forbidden

Pengguna tidak memiliki peran yang sesuai. Rute produk hanya untuk admin, sedangkan rute POS untuk admin dan kasir. Periksa kolom `role` pada tabel `users` dan alias middleware `role`.

5.5 Koneksi MySQL ditolak

Pastikan MySQL XAMPP berjalan dan kredensial di `.env` sesuai. Untuk XAMPP, jalankan `C:\xampp\mysql\bin\mysqld.exe` atau gunakan XAMPP Control Panel.

5.6 Stok tidak berkurang setelah transaksi

Periksa apakah transaksi benar-benar selesai tanpa exception. Jika ada exception di dalam `DB::transaction()`, seluruh perubahan memang dibatalkan. Lihat `storage/logs/laravel.log` untuk detail kesalahan.

5.7 Nomor invoice duplikat

Pola `count() + 1` tidak aman untuk transaksi bersamaan. Tambahkan unique index pada `invoice_number` (sudah ada di migrasi) dan tangani retry, atau gunakan sequence/UUID.

BAB 6

Daftar Berkas

Seluruh berkas berikut berada di dalam folder proyek `C:\xampp\htdocs\FPW\FPW_Test1`.

No	Berkas	Status	Keterangan
1	app/Http/Controllers/ProductController.php	Diubah	Search, filter, pagination, show, update, destroy
2	app/Http/Requests/UpdateProductRequest.php	Diubah	Authorize admin + unique pengecualian
3	resources/views/master-data/product/index.blade.php	Diubah	Form pencarian, filter, aksi lengkap
4	resources/views/master-data/product/edit.blade.php	Diubah	Form edit lengkap dengan old()
5	resources/views/master-data/product/show.blade.php	Dibuat	Detail produk + riwayat
6	app/Http/Controllers/PosController.php	Diubah	index, store, struk
7	app/Http/Requests/StorePosRequest.php	Dibuat	Validasi item keranjang
8	resources/views/pos/index.blade.php	Dibuat	UI kasir + Alpine.js
9	resources/views/pos/struk.blade.php	Dibuat	Struk dan tombol cetak
10	resources/js/app.js	Diubah	Import dan start Alpine
11	resources/css/app.css	Diubah	Aturan @media print
12	routes/web.php	Diubah	Rute pos.struk + perapian rute
13	package.json, package-lock.json	Diubah	Dependensi alpinejs

STRUKTUR FOLDER BARU

`resources/views/pos/` berisi `index.blade.php` dan `struk.blade.php`. Folder ini dibuat otomatis saat berkas pertama ditulis.

BAB 7

Referensi

1. Laravel 13 Documentation — Routing: Route Model Binding.
2. Laravel 13 Documentation — Eloquent: Relationships dan Collections.
3. Laravel 13 Documentation — Validation: Form Request dan wildcard rules.
4. Laravel 13 Documentation — Database: Getting Started (DB::transaction).
5. Laravel 13 Documentation — Vite dan frontend assets.
6. Alpine.js Documentation — x-data, x-for, x-model, dan reactive state.
7. Modul Praktikum FPW Pertemuan 11 — Detail Produk dan Riwayat Penjualan per Produk.
8. Modul Praktikum FPW Pertemuan 12 — Transaksi Penjualan (Point of Sale): Keranjang dan Simpan Transaksi.

PENUTUP

Dengan selesainya Pertemuan 11 dan 12, siklus inti POS Barokah Mart sudah berjalan: data master → pencarian → detail produk → transaksi → struk. Pertemuan 13 dan 14 tinggal menambahkan laporan penjualan, ekspor Excel, dashboard analitik, dan deployment.